Operimus Marketing Automation Platform - Report

Universidade de Aveiro(*UA*)

David Poeta Pelicano,
Guilherme Duarte Alves,
Jorge Guilherme Conceição Domingues,
João Pedro Ferreira Monteiro,
Henrique Manuel Pereira Ferreira

Version 1

Operimus - Marketing Automation Platform

Software Engineering

DETI - Departamento de Electrónica, Telecomunicações e Informática

David Poeta Pelicano,
Guilherme Duarte Alves,
Jorge Guilherme Conceição Domingues,
João Pedro Ferreira Monteiro,
Henrique Manuel Pereira Ferreira

(113391) davidpoetapelicano@ua.pt
(103185) guilherme.alves@ua.pt,
(113278) jorgeguilherme@ua.pt,
(114547) joaomonteiro29@ua.pt
(113600) ferreira.manuel.henrique04@ua.pt

Índice

1 Introduction **1**
1.1 Project Objectives . . . 1
1.2 Core Functionalities and Scope . . . 1

2 Personas and Access Management **1**
2.1 Personas . . . 1

3 Keycloak Configuration for Authentication and Authorization **2**

4 Role-based access control **2**

5 User Stories **4**
5.1 Content Creation . . . 4
5.2 Campaign Management . . . 4
5.3 Analytics and Dashboards . . . 5
5.4 Lead Exploration and CRM Sync . . . 5

6 Architecture **6**
6.1 Client Layer . . . 6
6.2 Load Balancing Layer . . . 7
6.3 Application Layer . . . 7
6.3.1 Backend Services . . . 7
6.4 Data Access Layer . . . 7
6.5 Identity and Access Management . . . 7
6.6 External Integrations . . . 8
6.7 Infrastructure Layer . . . 8
6.7.1 Observability Stack (Elastic Cloud) . . . 8
6.8 Sequence Diagram . . . 8

7 Conversational Assistant **8**

8 Frontend **10**
8.1 Project Structure . . . 10
8.2 Core Technologies . . . 10
8.3 Styling and UI Layer . . . 11
8.4 Feature-Specific Libraries . . . 11
8.5 Cross-Cutting Capabilities . . . 11

9 Backend **11**
9.1 Key Business Logic . . . 11
9.1.1 Workflow Engine . . . 11
9.1.2 Event-Driven Node Handlers . . . 12
9.1.3 Scheduled Services . . . 12
9.2 Domain Model . . . 12
9.3 Main Entities . . . 13
9.3.1 User . . . 13
9.3.2 Campaign Management . . . 13
9.3.3 Marketing Materials . . . 14
9.3.4 Workflow Automation . . . 14
9.3.5 Analytics and Reporting . . . 14
9.4 Swagger Backend Documentation . . . 14

10 Developer Onboarding **15**
10.1 Requirements . . . 15
10.2 Environment Setup & Installation . . . 15
10.2.1 Install Git . . . 15
10.2.2 Install Docker . . . 15
10.2.3 Install Maven . . . 16
10.2.4 Install npm . . . 16

10.2.5 Install Java . . . 16
10.2.6 Install Terraform . . . 16
10.2.7 Clone the Repository . . . 16
10.2.8 Configure Terraform Environment Variables: `terraform.tfvars` . . . 17
10.2.9 Configure Docker Environment Variables: `.env` . . . 17
10.2.10 Run shell script to setup machine IP . . . 18
10.2.11 Run application with Terraform . . . 18

11 Tests **18**
11.1 Unit Testing . . . 18
11.2 Integration Testing . . . 19
11.3 End-to-End (E2E) Testing . . . 19

12 Production Configuration **19**
12.1 Environmental Variables . . . 19
12.2 Build Process . . . 20
12.3 CI/CD Pipeline Architecture . . . 21
12.3.1 Continuous Integration (CI) . . . 21
12.3.2 Test Reporting and Observability . . . 21
12.3.3 Continuous Deployment (CD) . . . 21
12.4 Infrastructure Notes . . . 21

13 Feature Flag Management with Flagsmith **22**

14 Observability **22**
14.1 Backend Observability . . . 22
14.2 Frontend Observability . . . 23
14.3 SLOs/SLIs . . . 23

15 Development Workflow and Code Review Policies **24**
15.1 Backlog Organisation . . . 24
15.2 Pull Requests and Templates . . . 24
15.3 Code Review Rules . . . 25
15.4 CI/CD Validation . . . 25
15.5 Final Merge Policy . . . 25

List of Figures

1 Final System Architecture 6
2 Sequence diagram demonstrating the creation and import of workflow templates. 9
3 Chatbot System Architecture 10
4 Domain Model 13
5 Service Level Objectives 24

1 Introduction

This project, developed within the scope of the Software Engineering (ES) course, focuses on the design and specification of a **Marketing Automation Platform (MAP)**. In an era where personalized customer communication and operational efficiency are paramount, businesses require robust tools to manage the complexity of multi-channel marketing. The proposed MAP is a comprehensive software application designed to address these needs by streamlining, automating, and measuring marketing campaigns and tasks.

1.1 Project Objectives

The primary purpose of the platform is to help businesses effectively nurture leads and improve marketing efficiency at scale. By automating repetitive operational tasks, such as email transmissions, social media scheduling, and lead scoring, the MAP enables marketing teams to concentrate on high-value activities like strategy formulation and content creation.

The system is designed to serve a diverse range of professionals within marketing and sales departments, including:

- **Marketing Managers:** To design workflows and analyze campaign ROI.
- **Content Marketers:** To manage assets and engagement.
- **Sales Representatives:** To receive real-time intelligence on "hot" leads.
- **Marketing Analysts:** To interpret data and optimize performance.

1.2 Core Functionalities and Scope

As detailed in the system specification, the MAP includes a centralized hub for managing customer data and driving growth through targeted interactions. The core functionalities defined for this project include:

- **Lead Management and Segmentation:** Capturing and scoring leads based on behavior.
- **Automated Campaign Workflows:** Creating multi-step journeys with visual builders.
- **Content Management:** Tools for Email Marketing, Social Media Management, and Landing Pages.
- **Analytics and Integration:** Comprehensive reporting dashboards and seamless integration with CRM systems.

Beyond the technical requirements and architectural design, this project highlights how information systems engineering can significantly enhance marketing efficiency, support data-driven decision-making, and ultimately improve overall business performance.

2 Personas and Access Management

2.1 Personas

During the design of the Operimus Marketing platform, four primary personas were defined to represent the different types of users who interact with the system. Each persona embodies a specific set of responsibilities, objectives and operational needs, which are reflected in the permissions assigned within the application.

Marketing Manager

The Marketing Manager oversees the global coordination of marketing campaigns and the definition of automated strategies. This persona can create, edit and delete campaigns, build complete workflows, configure actions and delays, manage campaign materials and define segmentation rules and lead scoring criteria. In addition, the Marketing Manager can create reusable workflow templates that serve as the foundation for multiple campaigns. The Marketing Manager is also able to view the analytical reports generated by the Marketing Analyst, ensuring alignment between campaign execution and data-driven decision making.

Content Marketer

The Content Marketer is responsible for producing the creative materials used in campaigns. This persona can create, edit and delete Email Templates, Landing Pages and Forms. They can also schedule and manage social media posts and view or delete workflow templates. Their work focuses on content creation that will later be used by other roles throughout the campaign lifecycle.

Marketing Analyst

The Marketing Analyst focuses on evaluating metrics and analyzing campaign performance. This persona can build custom dashboards, select relevant analytical components and generate automatic PDF reports. Within the dashboard, the analyst can assess post performance, identify high-performing content, analyse lead progression within workflows and review statistics from form submissions associated with each campaign.

Sales Representative

The Sales Representative works directly with leads and potential customers. The platform allows this persona to access the list of leads, open individual lead profiles and review detailed information such as personal data, lead score and the complete interaction timeline. This timeline includes form submissions, landing page interactions and emails sent by the platform, organised across different categories to support efficient evaluation and follow-up actions.

3 Keycloak Configuration for Authentication and Authorization

To ensure centralised authentication and access control, Keycloak was adopted as the platform's Identity and Access Management system. The application operates within a dedicated realm named *marketing-realm*, whose full configuration is stored in the `marketing-realm.json` file. This file specifies all relevant parameters related to users, permissions, roles and authentication clients.

The realm is active and configured to allow user registration, password reset and email-based login. The use of duplicate email addresses is disabled to ensure data consistency. Four main realm roles are defined, directly corresponding to the personas described earlier: `MARKETING_MANAGER`, `CONTENT_MARKETER`, `MARKETING_ANALYST` and `SALES_REPRESENTATIVE`.

The configuration also includes two essential clients. The `marketing-backend` client is confidential, secured with a client-secret and used for internal service-to-service communication. The `marketing-frontend` client is a public client used by the React application, enabling the OAuth login flow and redirect handling in the browser. Both clients define the authorised callback URLs and web origins required for the development environment.

Finally, the realm includes predefined test users, each assigned to one of the roles, supporting the validation of access permissions during development. This configuration enables the system to implement a coherent Role Based Access Control model fully aligned with the defined personas, ensuring that each user has appropriate access based on their functional responsibilities within the platform.

4 Role-based access control

The RBAC model assigns a fixed set of permissions to each persona. These permissions determine which resources users can access and what operations they can perform. Table 1 summarizes the access model enforced by the frontend routes, component-level guards, and backend administrative endpoints.

Notes

- **Frontend route guards:** Protected routes are implemented with a global protected-route component (see `proj/frontend/src/components/ProtectedRoute.tsx`) and per-route `requiredRoles` defined in `proj/frontend/src/App.tsx`.
- **Component-level gating:** Many UI actions (create/delete/edit buttons) are additionally hidden or disabled using `hasRole(...)` checks inside cards and builders (e.g., `CampaignCard`, `EmailTemplateCard`, `LandingPageCard`, `WorkflowBuilder`).
- **Backend admin endpoints:** Administrative API paths under `/api/admin/*` are restricted to the `MARKETING_MANAGER` role (see `SecurityConfig.java` in the backend).

Page	**Manager**	**Content**	**Analyst**	**Sales**
List Campaigns	×	×	×	×
View Campaign data	×			
Create Campaign	×			
List Email Templates	×	×	×	×
View Email Template	×	×	×	×
Create Email Templates		×		
Edit Email Templates			×	
List Landing Pages	×	×	×	×
View Landing Page	×	×	×	×
Create Landing Pages		×		
Edit Landing Pages		×		
View Social Posts Schedule	×	×	×	×
Create Social Posts		×		
View Social Post Data		×		
List Forms	×	×	×	×
View Form	×	×	×	×
Create Form		×		
Edit Form		×		
List Workflows	×	×	×	×
Create Workflow-Template	×			
Edit Workflow-Template	×			
List Workflows	×	×	×	×
Create Workflows	×			
Edit Workflow	×			
List Dashboards	×	×	×	×
Create Dashboards			×	
Edit Dashboards			×	
View Dashboards			×	
List Reports	×	×	×	×
View Report			×	
List Leads	×	×	×	×
View Lead Information			×	
Public: Fetch Form Templates	Public — no authentication required			
Public: Fetch Landing Pages	Public — no authentication required			
Public: Submit Form (POST)	Public — no authentication required			

Table 1: Role Based Permission matrix.

- **Public endpoints:** Form-template and landing-page read endpoints and form-submission POST requests are exposed without authentication (via CORS and `permitAll` rules in the security configuration).

5 User Stories

5.1 Content Creation

The platform supports several types of marketing assets, including email templates, landing pages, forms and social posts. All these assets follow a common permission model: only the `CONTENT_MARKETER` role can create, edit and delete them, while other personas have view-only access. This ensures consistent control over all campaign materials throughout their lifecycle.

Regardless of their type, content assets are stored in the backend together with their metadata, allowing them to be searched, previewed, reused and associated with campaigns when required. Access control is enforced according to the role definitions configured in Keycloak.

Create and save Email Templates

Email templates allow the Content Marketer to design personalised communications to be used across different campaigns. Templates include a name, subject, description and the final HTML body, and support merge tags for dynamic personalisation. They can be previewed, updated and reused whenever necessary.

Create Landing Pages

Landing pages serve as focused conversion points for campaigns, providing a dedicated page where visitors are encouraged to perform a specific action, such as submitting a form or engaging with campaign material. Each landing page includes a name, optional description and a fully customisable layout, and can later be linked to a campaign or reused in future initiatives.

Create Form Templates Forms provide a structured way to collect user input within marketing campaigns. The Content Marketer can add different types of fields (text inputs, dropdowns, ratings, checkboxes, file uploads, etc.), configure validation rules, and design the logical flow of the form.

Each form template includes a name, optional description and a structured JSON representation that defines all fields and logic. This representation is stored in the backend together with its metadata, allowing templates to be previewed, modified and reused across campaigns.

Schedule post for social media Social posts allow campaign content to be published or scheduled directly to social media. The Content Marketer can define the post's name and description, upload an image and select the intended publication mode, either immediate or scheduled for a future date and time.

The creation flow includes support for selecting target platforms; however, in the current version of the system, Facebook is the only platform fully integrated with the backend publishing workflow. The interface also provides client-side validation for scheduled posts and a confirmation dialog to prevent accidental publication.

Each post is stored in the backend together with its metadata, including the chosen publication time, selected platform and uploaded media file. Posts can later be viewed or associated with campaigns as part of the broader content strategy.

5.2 Campaign Management

Create and Manage Campaigns

Campaigns serve as the central organisational unit of the marketing platform, grouping together workflows, content assets and scheduling logic for specific marketing initiatives. The Marketing Manager can create new campaigns by defining a name, description and high-level objectives, after which campaigns become available for configuration and association with content such as email templates, forms, landing pages and social posts. In addition to creating campaigns, the Marketing Manager is also responsible for adding these materials to each campaign, ensuring that all assets needed for automation and execution are properly organised.

The campaigns overview page provides searching, filtering and management capabilities. Campaigns can be sorted by status (Active, In Progress or Finished), searched by name and inspected individually. The system allows the Marketing Manager to delete campaigns, with a confirmation modal preventing accidental removal. All campaign data is stored in the backend together with its status and metadata, enabling campaigns to be retrieved, updated and reused as part of broader automation strategies.

Automated Marketing Workflows

The platform allows users to design automated workflows that coordinate multi-step marketing actions inside a campaign. A workflow defines how leads are processed over time through actions such as sending emails, applying delays, checking conditions, or updating lead data.

Users can create workflows from scratch or import reusable templates, allowing faster setup and consistent automation patterns across campaigns. After creation, the workflow can be customised by adding nodes, adjusting logic, and connecting steps to match the campaign's goals.

The system provides validation, confirmation steps, and clear feedback during the creation process, ensuring reliability. Workflow creation is also instrumented with observability to monitor user actions and API performance.

In essence, automated workflows give marketing teams a structured way to build scalable, repeatable, and data-driven campaign automation.

5.3 Analytics and Dashboards

Select which data appears in dashboard This user story introduced a flexible and fully configurable campaign dashboard, enabling users to decide which types of analytical information should be displayed for each campaign. A diverse set of metrics and performance indicators was made available for selection, including form submission statistics, lead quality distribution, social post performance, post-level engagement metrics, workflow status, and other key campaign indicators. These elements can now be combined freely, allowing each dashboard to reflect the specific analytical needs of the campaign it represents.

The dashboard supports personalised layouts, where users may insert, remove, organise, and visualise different widgets according to their goals. Each campaign retains its own customised dashboard, ensuring that the selected data sources and visual elements remain consistent across sessions and accessible to all relevant stakeholders.

Only the *Marketing Analyst* is permitted to create and edit dashboards, while all other personas retain the ability to view the dashboards associated with their campaigns.

In addition, social media metrics and rankings of top-performing posts (introduced in later user stories) can be included as optional dashboard widgets, allowing analysts to integrate engagement insights directly into their custom analytical views.

Generate and Export Campaign Reports

The platform provides a complete reporting system that allows teams to capture, store, and retrieve analytical snapshots of each marketing campaign. The Marketing Analyst can generate detailed PDF reports that reproduce the exact state of the campaign dashboard, including performance metrics, social post analytics, lead quality, form statistics, workflow information, and trend visualisations. Each generated report is automatically archived and linked to its campaign, creating a historical record of performance over time.

Users with strategic roles, such as the Marketing Analyst and the Marketing Manager, can browse all previously generated reports through a dedicated interface that groups documents by campaign and displays essential metadata such as creation timestamps. Individual reports can be downloaded for sharing, review, or long-term storage, supporting data-driven decision-making and transparent communication with stakeholders.

5.4 Lead Exploration and CRM Sync

Lead Management

The platform offers a comprehensive interface for browsing, filtering and analysing leads collected across marketing campaigns. Users can search leads by name, email or country, and refine the results using status-based filters (Hot, Warm, Cold), witch automatically derived from the lead scoring model.

Each lead card displays key information such as contact details, country, score and associated segmentation labels, enabling marketing and sales teams to quickly assess lead relevance and priority. A dedicated lead view is available to Sales Representatives, providing full visibility into a lead's interaction history (e.g., received/opened emails, submitted forms, and viewed landing pages).

The module consolidates all leads into a single, structured view and provides dynamic filtering, segmentation awareness and status visualisation. Together, these capabilities ensure that teams can efficiently work with the most relevant lead information and maintain a clear overview of audience characteristics across campaigns.

Sync Lead Information with a CRM System

Additionally, an integration with a Customer Relationship Management (CRM) platform was implemented to ensure full two-way synchronization between our system and an external CRM environment. This integration enables leads to be managed in a more professional, sales-oriented context while maintaining consistency between both systems. For this purpose, HubSpot was selected as the CRM provider due to its robust API, extensive automation capabilities, and suitability for marketing-driven workflows.

The synchronization process operates bidirectionally. Whenever a lead is created, updated, or removed within our platform, the backend propagates these changes to HubSpot, ensuring that the CRM dashboard always reflects the most recent information. Conversely, modifications performed directly within HubSpot,

such as updates to lead attributes or status, can be detected by the platform. When such changes occur, the frontend prompts the user to synchronize the leads page, ensuring that the local state remains aligned with the CRM system.

To support this integration, two additional properties were introduced in HubSpot: *score* and *status*. The status field categorises leads as **COLD**, **WARM**, or **HOT**. These fields mirror the structure of the Lead entity within our system and ensure that both platforms share a unified representation of lead relevance and priority.

Overall, this integration strengthened the system's marketing automation capabilities by ensuring that all lead-related information remains consistent across platforms, reducing manual effort and improving the reliability of customer data throughout the workflow.

6 Architecture

The architecture was one of the elements of this project which suffered more changes with its development, since the introduction of more features meant the introduction of new elements to fulfill those requirements. This section introduces the final architecture, while explaining its major components. Additionally, a sequence diagram was introduced which further explains the functioning of the system when a user performs one of the Use Cases that was implemented.

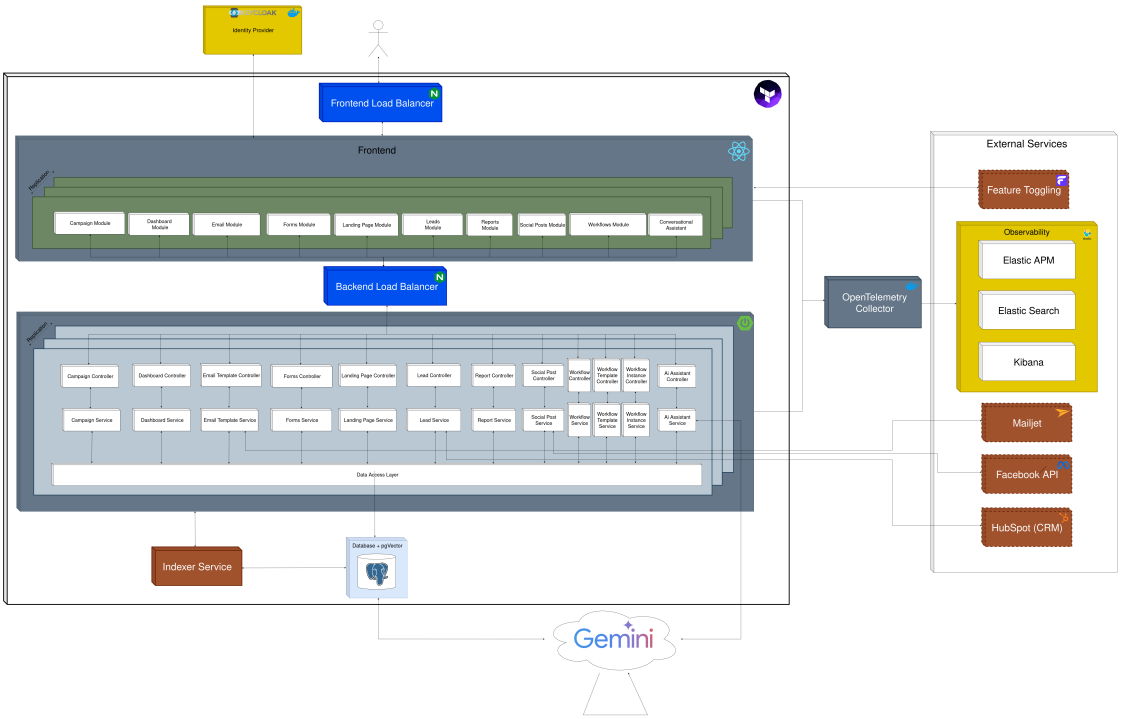

Figure 1: Final System Architecture

6.1 Client Layer

The client interface is implemented in React and delivered through Dockerized frontend containers. All user requests are routed through the NGINX frontend load balancer, ensuring that multiple frontend replicas can operate concurrently and handle increased demand.

The frontend contains several modular features, including:

- Campaign Management
- Lead and CRM views
- Social Media Management

- Workflow creation and editing
- Analytics and reporting dashboards
- Email template editor
- Landing page builder
- Conversational Assistant powered by LLMs

The frontend communicates exclusively with the backend API through the backend NGINX load balancer, ensuring isolation between layers and supporting horizontal scaling.

6.2 Load Balancing Layer

To guarantee reliability and support elastic scaling, the system employs two independent NGINX load balancers:

- **Frontend Load Balancer:** Distributes all incoming browser requests across available frontend containers.
- **Backend Load Balancer:** Exposes a unified API endpoint and forwards requests to multiple Spring Boot backend replicas. This component is essential for high availability and enables rolling upgrades with zero downtime.

Both load balancers are provisioned and configured using Terraform, ensuring consistent infrastructure management across deployments.

6.3 Application Layer

6.3.1 Backend Services

The backend is implemented in Spring Boot and organised into domain-driven modules that encapsulate the system's core functionalities. These modules cover all major areas of the platform, including campaign management, lead and CRM operations, social media scheduling and engagement analytics, the workflow engine for template and instance execution, analytics and reporting, email template management, form and landing page generation, and the conversational assistant powered by AI.

Each module follows a layered internal architecture consisting of: **Controllers**, responsible for handling HTTP requests, **Services** encapsulating business logic and **Repositories/Data Access** abstracting persistence logic.

In addition to business logic, the backend integrates directly with several external services, which will be mentioned later on.

6.4 Data Access Layer

The system relies on PostgreSQL as its primary data store, extended with `pgVector` to enable embedding-based search and retrieval.

A dedicated **Indexer Service** processes documents, computes vector embeddings, and stores them in the pgVector index. This component supports advanced AI-driven features, such as contextual retrieval for the Conversational Assistant.

This layered design keeps persistence concerns isolated from backend modules, ensuring clean separation of business logic and storage mechanisms.

6.5 Identity and Access Management

User authentication and authorization are handled by Keycloak, which is deployed through Docker Compose and integrated with the services provisioned by Terraform. Keycloak provides OAuth2/OpenID Connect authentication, supports role-based access control, and enables secure token introspection for backend services. In addition, it ensures consistent propagation of user identity throughout the system, including in components that rely on personalised feature flagging.

6.6 External Integrations

The platform interacts with several third-party systems through dedicated backend clients:

- **Flagsmith**: used for feature release management and A/B testing.
- **MailJet**: handles email delivery.
- **Facebook Graph API**: enables creation and monitoring of social media posts, providing engagement metrics.
- **HubSpot CRM**: builds a bidirectional relation with the system, that synchronises lead and contact data.
- **Google Gemini**: provides system context and tools to create new entities

6.7 Infrastructure Layer

The infrastructure layer is fully containerised and orchestrated through a hybrid model that combines Terraform and Docker Compose. Terraform is responsible for provisioning the core infrastructure components, including the Docker networks, persistent volumes, backend and frontend replicas, the NGINX load balancers, and the PostgreSQL database.

In parallel, Docker Compose is used to manage auxiliary services that complement the core infrastructure. Specifically, it is responsible for deploying the Keycloak identity provider and the OpenTelemetry Collector, both of which operate alongside the Terraform-provisioned components.

6.7.1 Observability Stack (Elastic Cloud)

To ensure full visibility into system behaviour, the platform integrates a comprehensive observability pipeline consisting of:

- **OpenTelemetry Collector**: receives OTLP logs, traces, and metrics from frontend and backend.
- **Elastic APM**: visualises distributed traces.
- **Elasticsearch**: stores logs and metrics.
- **Kibana**: provides dashboards, SLO visualisations, and alerting.

This architecture enables end-to-end trace correlation, real-time health monitoring, and performance analysis across all system components.

6.8 Sequence Diagram

To illustrate how the components interact in a real user flow, a sequence diagram was created, as seen in 2. It demonstrates the process of creating a workflow template and later importing it into a campaign workflow.

7 Conversational Assistant

The conversational assistant in this project is designed to provide context-aware responses, enable semantic exploration of marketing assets, and offer guided assistance for creating or modifying campaigns, templates, landing pages, email campaigns, posts and more via the chatbot. The platform incorporates an AI-assisted interaction layer based on a Retrieval-Augmented Generation (RAG) approach with agentic capabilities, delivering a conversational interface that transforms the system into an intelligent assistant capable of supporting marketers during planning, configuration, and analysis tasks. The AI component combines semantic retrieval of relevant entities using `pgvector`, Google Gemini models for natural-language generation and reasoning, conversational memory to maintain consistent multi-turn interactions, and tool integrations that allow the AI agent to perform CRUD operations when instructed.

The system architecture, illustrated in Figure 3, consists of a React frontend that communicates with a Spring Boot backend. The frontend includes the Assistant UI, a dedicated chat interface that captures user input, manages API calls, renders responses with visual feedback, and maintains conversation state across multiple interactions. When a user submits a question through the Assistant UI, the request is sent to the backend, where the agent service performs a similarity search on stored vectors to identify relevant context.

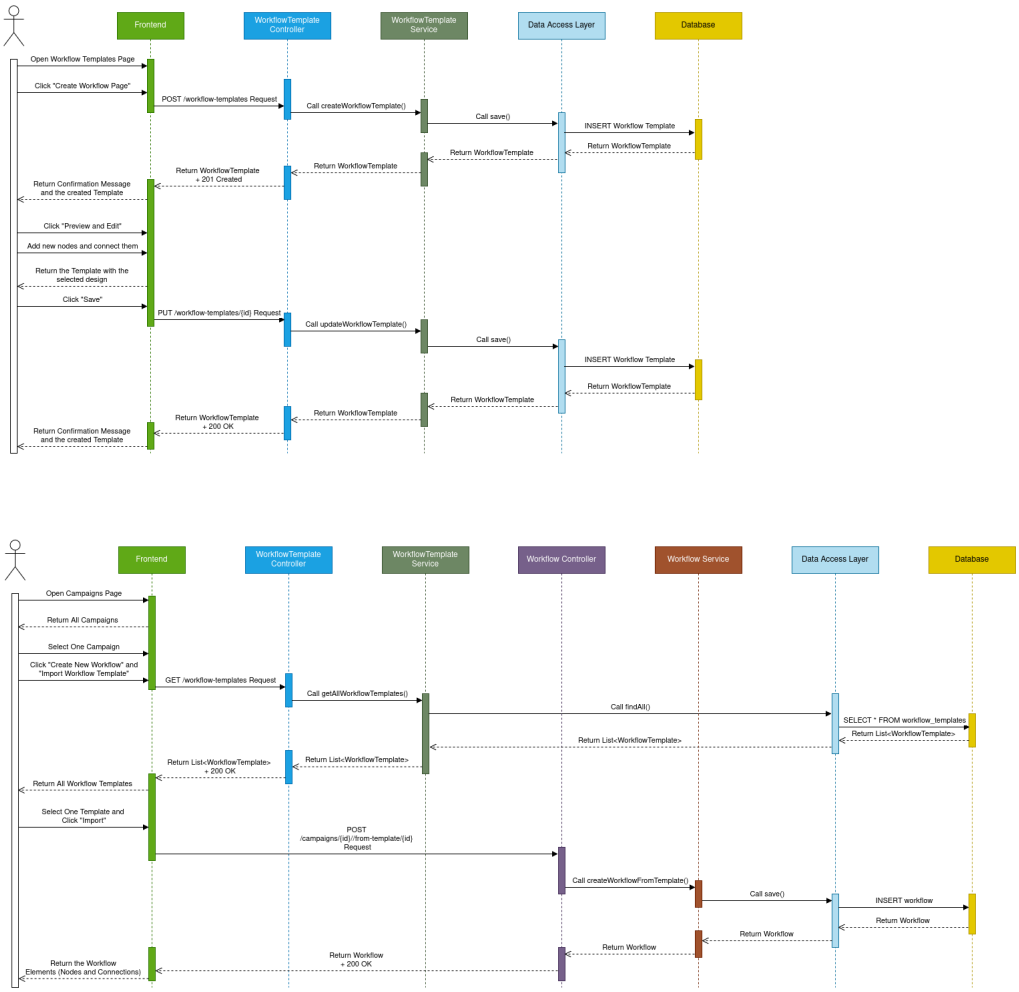

Figure 2: Sequence diagram demonstrating the creation and import of workflow templates.

It then constructs an enriched prompt that is passed to the large language model, **Gemini 2.0 Flash**, which generates context-aware answers, enables semantic exploration of marketing assets, and provides guided assistance through the chatbot, deciding whether to respond directly or invoke a tool to perform system actions.

Supporting the RAG subsystem is the Vector Indexing Service, which is responsible for embedding and indexing marketing entities such as campaigns, templates, forms, landing pages, posts, and leads. Whenever an entity is created or updated, the service generates a cleaned textual representation, produces embeddings using the AI model, and stores them in the vector database. This automatic indexing mechanism ensures that semantic search remains consistent and up to date, enabling the AI layer to reason reliably over the latest content in the system. The backend also includes an indexing mechanism that transforms raw data into vectors for retrieval, ensuring that the agent can access up-to-date information efficiently. All conversation history is stored in the database to maintain context across interactions, while configuration parameters separate production and testing environments to provide isolation for integration tests. The complete execution flow results in the generated response being returned to the frontend, where it is displayed in the Assistant UI. This architecture ensures that the system can handle complex queries, maintain conversation state, and extend functionality through callable tools, providing a robust and scalable conversational assistant.

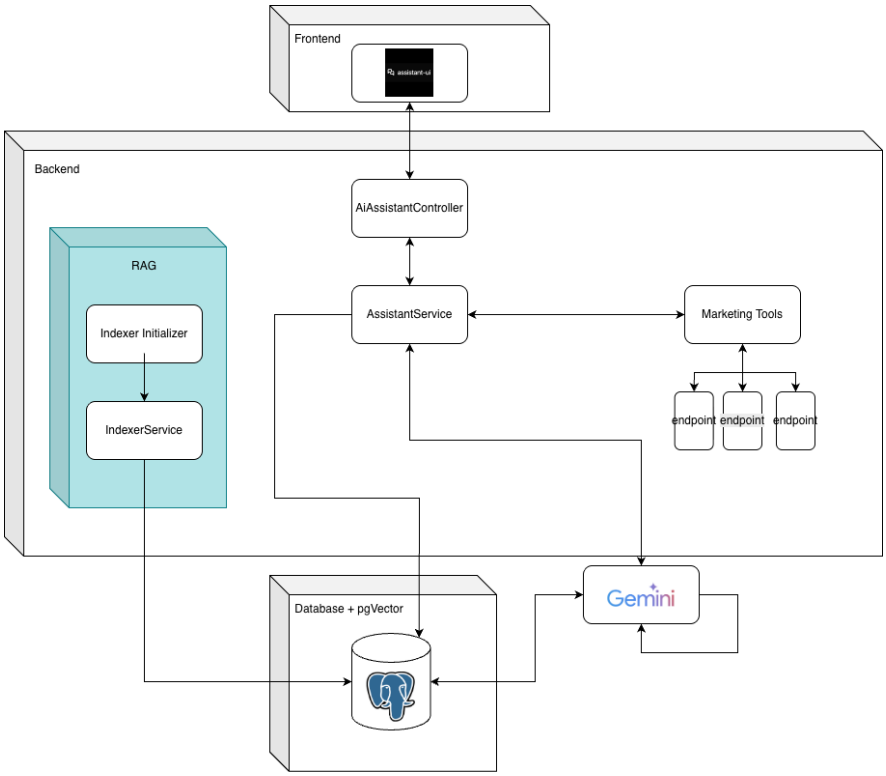

Figure 3: Chatbot System Architecture

8 Frontend

The frontend of the Operimus Marketing platform is designed to be fast, modular and scalable, supporting content creation, campaign analytics and workflow automation. It is built using **React**, **TypeScript**, and **Vite**, a stack chosen for predictable behaviour, maintainability and high runtime performance.

8.1 Project Structure

The `src/` directory is organised for long-term maintainability, including:

- **API modules**
- **Reusable components**: alerts, buttons, graphics, cards, messages, modals, filters, etc.
- **Pages**: campaigns, posts, forms, dashboards, workflows, landing pages, reports.
- **React contexts**, custom hooks, utility functions.
- **Type definitions** and **observability modules**.

8.2 Core Technologies

- **React** – Provides a component-based architecture, supporting reusable and declarative UI elements. This enables complex modules such as:
  - Email editor
  - Form builder
  - Analytics dashboards
  - Workflow builder
- **TypeScript** – Adds static typing to reduce runtime errors and simplify refactoring.
- **Vite** – Supplies a very fast dev server and highly optimised production builds.

8.3 Styling and UI Layer

- **Tailwind CSS** for utility-first styling and fast creation of responsive layouts.
- **DaisyUI** for configurable UI components when more structured styling is needed.
- **ESLint** and **Prettier** to ensure consistent code style and quality.

8.4 Feature-Specific Libraries

- **XYFlow** – Powers the visual workflow builder with draggable nodes and connections.
- **SurveyJS** – Implements the form builder and dynamic rendering of forms and landing pages.
- **React Email Editor** – Provides a drag-and-drop interface for creating email-compatible HTML.
- **Chart.js** and **Recharts** – Drive dashboard visualisations.
- **React Grid Layout** – Used for creating movable, resizable dashboard widgets.

8.5 Cross-Cutting Capabilities

- **Keycloak JS** – Handles authentication, sessions and role enforcement via OpenID Connect.
- **OpenTelemetry** – Collects frontend traces and metrics for performance and observability.
- **Flagsmith** – Manages feature flags, staged rollouts and selective feature activation.
- **Axios-based API modules** (in `src/api`) – Centralise all backend interactions and prevent HTTP calls from UI components.

9 Backend

The backend of the Operimus Marketing Platform provides the core application logic that enables campaign automation, workflow orchestration, customer segmentation, AI-assisted interactions and integration with external marketing services. It is implemented using Spring Boot and follows a layered architecture composed of controllers, services, repositories and domain entities. The backend exposes a REST API consumed by the frontend application, enforces authentication and authorization through Keycloak, manages the persistence of marketing data in PostgreSQL and executes asynchronous and scheduled tasks that support automation flows.

Beyond traditional request handling, the backend incorporates a workflow engine, a semantic indexing pipeline and an AI-powered RAG subsystem that together form the intelligent foundation of the platform. These components allow the system to analyse content, react to user behaviour, automate marketing actions and provide contextual assistance to end users. The following subsections describe the key elements of this business logic.

9.1 Key Business Logic

The backend of the Operimus Marketing Platform implements several core business mechanisms that extend the system beyond traditional CRUD operations. These components form the intelligence layer of the application, supporting workflow automation, semantic search and asynchronous processing. This section describes the main subsystems that constitute the core business logic of the platform.

9.1.1 Workflow Engine

At the centre of the automation capabilities lies the *Workflow Engine*, responsible for orchestrating the execution of marketing workflows associated with campaigns. It processes domain events such as form submissions, email interactions or landing-page visits, evaluates the applicable workflow triggers and advances each lead through its corresponding workflow.

The engine manages the creation and progression of *Workflow Instances*, ensuring that each lead follows the correct path according to the workflow structure and campaign state. Its architecture is designed to be extensible and modular, allowing new workflow behaviours to be added without impacting existing functionality.

9.1.2 Event-Driven Node Handlers

Workflow execution relies on a strategy-based mechanism implemented through specialised *Node Handlers*. Each handler encapsulates the logic of a specific workflow action, enabling both synchronous and asynchronous processing. Representative examples include sending marketing emails, applying delays, modifying segment membership or reacting to incoming events.

This pattern provides strong separation of concerns and facilitates the introduction of new workflow actions simply by defining additional handler implementations. It also supports an event-driven execution model, in which workflows respond dynamically to user behaviour and external system activity.

9.1.3 Scheduled Services

The backend includes several scheduled services that support asynchronous processing and integration with external APIs:

- **Delay Scheduler** executes actions scheduled within workflows at their designated timestamps;
- **Post Scheduler** publishes social media content at predefined times;
- **MailJet Polling Service** periodically ingests email engagement events, keeping the platform synchronised with the email provider.

These background services enable the platform to maintain continuous automation flows, ensuring that workflows progress correctly and that external interactions are reflected in real time.

9.2 Domain Model

The platform's domain model 4 represents the core structure of a Marketing Automation Platform (MAP), describing the main entities, their attributes, and the relationships between them. It captures how users, marketing resources, campaigns, and analytics interact within the system.

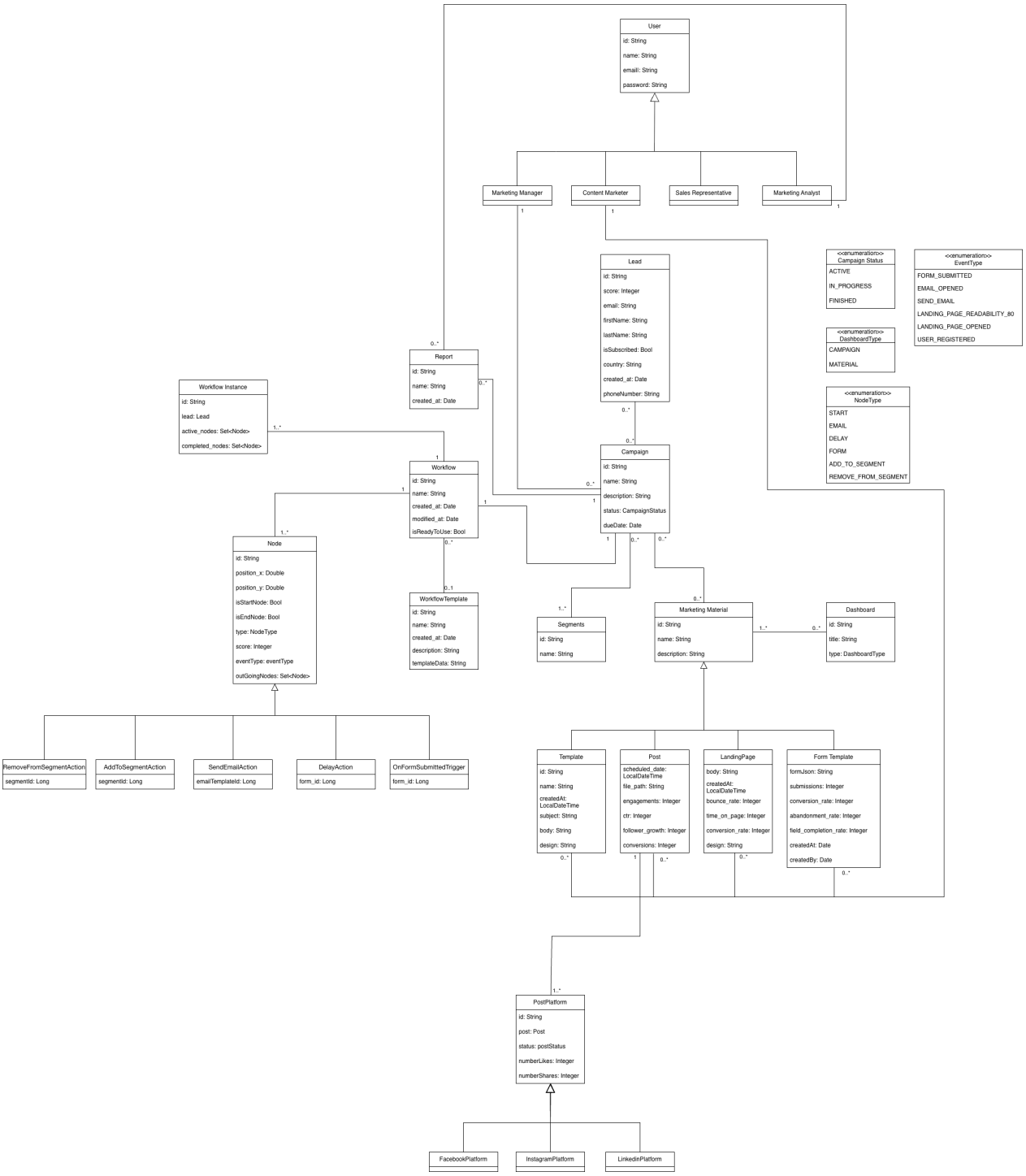

Figure 4: Domain Model

9.3 Main Entities

9.3.1 User

The **User** entity defines the basic authentication and profile data for all users. Including the specialized roles, Marketing Manager, Content Marketer, Sales Representative, and Marketing Analyst.

9.3.2 Campaign Management

A **Campaign** acts as the central structure linking planning, execution, and evaluation. Its fields capture metadata such as name, timing, descriptive details, and lifecycle state using the `CampaignStatus` enumera-

tion. **Leads** contain identifying information, contact details, geographic attributes, engagement indicators, and subscription preferences. Their fields support tracking growth over time and reacting to interactions captured by the system.

Segments model logical groupings of leads, storing descriptive names and criteria used to filter or classify audiences. Campaign–segment relationships allow targeting strategies to be defined explicitly.

9.3.3 Marketing Materials

Marketing Material serves as an abstract parent entity that aggregates properties common to all content types, such as creation metadata, publication status, and descriptive attributes. Concrete subclasses refine these fields according to their medium:

- **Mail** adds fields for subject lines, body content, template references, and metrics such as opens or conversions.
- **Post** includes content fields, publication timestamps, platform-specific identifiers, and counters for impressions or user interactions.
- **Landing Page** extends material metadata with page structure, design data, visit counters, and conversion tracking.
- **Form Template** stores a serialised definition of form fields, validation rules, and linked automation triggers.

Platform entities (e.g., Facebook, Instagram, LinkedIn) hold configuration attributes and API integration details needed to publish posts programmatically.

9.3.4 Workflow Automation

Workflow represents an automated process tied to a campaign. It stores structural information such as name, description, last execution time, and the current phase or state. Workflows are composed of interconnected **Nodes**, each storing positional information for visual editors, node type descriptors, activation settings, and references to the operation it performs.

Reusable **Workflow Templates** mirror the structure of workflows but emphasise replication, storing descriptive fields and JSON-serialised representations of nodes and edges.

Workflow Instance objects track execution for individual leads, retaining attributes related to progress, active nodes, and branch decisions.

9.3.5 Analytics and Reporting

Report entities aggregate metric data derived from campaigns, workflows, and materials. Fields capture the reporting period, generation timestamps, descriptive labels, and numerical performance attributes.

Dashboard stores configuration layouts and a type indicating whether it displays campaign-wide metrics or material-specific KPIs. Its fields allow for flexible UI rendering and persistent customisation.

9.4 Swagger Backend Documentation

The backend provides interactive API documentation through **Swagger UI**, allowing developers to explore, test, and validate all available REST endpoints exposed by the Spring Boot application. This interface is automatically generated from the OpenAPI specification and offers a structured overview of request and response schemas, authentication requirements, and endpoint behaviour.

Swagger UI can be accessed locally at:

```
http://localhost:8080/swagger-ui/index.html
```

To authenticate within Swagger UI and test protected endpoints, use the following client credentials configured in Keycloak:

- **client_id**: `marketing-backend`
- **client_secret**: `my-secret`

These credentials allow Swagger UI to retrieve an access token via the OAuth2 client credentials flow, enabling the execution of secured API requests directly from the documentation interface.

10 Developer Onboarding

This section provides the essential prerequisites and setup steps required for contributors to start working with the backend service. It outlines the tools, dependencies, and environment configuration needed to build, run, and test the application consistently across development environments.

10.1 Requirements

This project requires the following tools and environments to ensure proper development, build, and deployment processes:

- **Java 17** (Temurin or OpenJDK recommended): Required to run the Spring Boot application and other Java-based components. Using Java 17 ensures compatibility with modern frameworks and long-term support.
- **Docker**: Allows packaging and running the application in isolated containers, simplifying deployment and ensuring consistency across different environments.
- **Maven**: Used for dependency management, project building, and running tasks such as testing and generating reports. It ensures reproducible builds.
- **Terraform**: Manages infrastructure as code, enabling automated provisioning and configuration of cloud resources required by the project.
- **Node.js and npm**: Node.js provides the runtime for JavaScript-based tools and frontend development. npm (Node Package Manager) is used to manage frontend dependencies, run scripts, and build the client-side application.

10.2 Environment Setup & Installation

This subsection provides the full set of installation steps required to prepare a development machine for running the application and its supporting infrastructure. Follow the instructions below to ensure all dependencies, tools, and environment variables are correctly configured.

Installation Instructions

10.2.1 Install Git

```
sudo apt update
sudo apt install -y git
```

10.2.2 Install Docker

```
# Add Docker's official GPG key:

sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
    docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
    compose-plugin

sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
```

10.2.3 Install Maven

```
sudo apt install maven
```

10.2.4 Install npm

```
sudo apt install npm
```

10.2.5 Install Java

```
sudo apt install openjdk-17-jdk
```

10.2.6 Install Terraform

```
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common

wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null

gpg --no-default-keyring \
--keyring /usr/share/keyrings/hashicorp-archive-keyring.gpg \
--fingerprint

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/
    hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP
    '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sudo tee
    /etc/apt/sources.list.d/hashicorp.list

sudo apt update
sudo apt-get install terraform
```

Setup Instructions

10.2.7 Clone the Repository

```
git clone https://github.com/detiuaveiro/group-project-es2526_301.git
cd group-project-es2526_301/proj
```

> **Important:** Both the `terraform.tfvars` and `.env` configuration files must be placed in the root of the `proj/` directory, as the deployment scripts and Docker/Terraform configurations expect them to be located there.
>
> **Note:** For convenience, a Markdown file containing all required environment variables is available at the following link. This helps ensure that long values such as API keys and access tokens are copied correctly: `https://uapt33090-my.sharepoint.com/:f:/g/personal/jorgeguilherme_ua_pt/IgB-hVD7x1fBQpxIP-ODKabkAWPrmZGLsFHIcwBdS8dNNVc?e=nUsZT9`

10.2.8 Configure Terraform Environment Variables: `terraform.tfvars`

```
cat <<'EOF' > terraform.tfvars
backend_docker_port = 8080
frontend_docker_port = 5173

spring_profiles_active  = "prod"
spring_security_issuer_uri = "http://localhost:8081/realms/marketing-realm"
spring_security_jwk_uri = "http://keycloak:8080/realms/marketing-realm/protocol/openid-connect/
    certs"

mailjet_api_username = "2ad8014e623870836f919fa12f9531f4"
mailjet_api_password = "2d488385677c01cfe92dad979760269d"

vite_backend_host = "http://localhost:8080"
vite_backend_port = "8080"
vite_keycloak_url = "http://localhost:8081"
vite_flagsmith_env_id = "9aXCbPY7WK3TzzKiNCsCDZ"
vite_api_version = "v3"

postgres_user = "user"
postgres_password = "password"
postgres_db = "operimusdb"
postgres_local_port = 5433
postgres_docker_port = 5432

backend_replicas  = 2
frontend_replicas = 2

frontend_lb_external_port = 3000
frontend_lb_internal_port = 80

backend_lb_external_port = 8080
backend_lb_internal_port = 80

otel_exporter_otlp_endpoint = "https://28a10e2de2204f84b982cf06472b3e41.apm.us-central1.gcp.cloud.
    es.io/"
otel_exporter_otlp_headers = "cURUQ19ab0JfaVByOGVKT3dFS1A6dG5WSk1raW96R1p6ZUJKOERtNzA3UQ=="

gemini_api_key = "AIzaSyBHlN4KXv7L9Vu2j0V_a4Pu1bLoZz_AAwQ"
google_project_id = "428306026048"

facebook_access_token = "EAAWBRYKIHdEBQHsGHAk6esPdozMeXDRJmFbSbg96OgrFoVPToTzJy8Bs6zojROrNS0
    IqNpYNtGTNKefJPgZBTnL4jDJZBM46AIwcPP303GHVYFogMPpmSmkVKtV7bOHspeYbe23BiZAkWzY2
    AmBBRSUTBNXYseVkON17SXFSQ2CBlo0oMtQE1mtw8jEwnNvELqfBIQdzV2ILZB8JZAscy"
facebook_app_id = "1549510426238417"
facebook_app_secret = "c3c8605a24e8c8410fff43799d07b0a5"
facebook_page_id = "879828281881355"
facebook_public_page_id = "61584053344153"

hubspot_access_token = "pat-eu1-332764ad-d6a9-4b0e-82cb-d3ca5e968e5d"

machine_url = "localhost"
EOF
```

10.2.9 Configure Docker Environment Variables: `.env`

```
cat <<'EOF' > .env
KEYCLOAK_IMAGE=quay.io/keycloak/keycloak:24.0.2
KEYCLOAK_ADMIN=admin
KEYCLOAK_ADMIN_PASSWORD=admin
KEYCLOAK_LOCAL_PORT=8081
KEYCLOAK_DOCKER_PORT=8080
KEYCLOAK_REALM=marketing-realm
```

```
KEYCLOAK_IMPORT_PATH=./keycloak/realms
KC_HOSTNAME_URL=http://localhost:8081

SPRING_PROFILES_ACTIVE=prod
SPRING_SECURITY_ISSUER_URI=http://localhost:8081/realms/marketing-realm
SPRING_SECURITY_JWK_URI=http://keycloak:8080/realms/marketing-realm/protocol/openid-connect/certs

BACKEND_LOCAL_PORT=8080
BACKEND_DOCKER_PORT=8080
SPRING_PROFILES_ACTIVE=prod
SPRING_SECURITY_ISSUER_URI=http://localhost:8081/realms/marketing-realm
SPRING_SECURITY_JWK_URI=http://keycloak:8080/realms/marketing-realm/protocol/openid-connect/certs

POSTGRES_DB=operimusdb
POSTGRES_USER=user
POSTGRES_PASSWORD=password
PG_LOCAL_PORT=5433
PG_DOCKER_PORT=5432

FRONTEND_LOCAL_PORT=5173
FRONTEND_DOCKER_PORT=5173
VITE_BACKEND_HOST=http://localhost:8080
VITE_BACKEND_PORT=8080
VITE_KEYCLOAK_URL=http://localhost:8081
VITE_UNLAYER_PUBLIC_KEY=PyDSrh4TSawHBJZhk3zK7GP1AJO2SwTw0Ryh2JnWAjpc9hcf6KJkbvWAMFxTD2y

MAILJET_API_USERNAME=2ad8014e623870836f919fa12f9531f4
MAILJET_API_PASSWORD=2d488385677c01cfe92dad979760269d

VITE_FLAGSMITH_ENV_ID=9aXCbPY7WK3TzzKiNCsCDZ
VITE_API_VERSION=v3
VITE_BACKEND_HOST=http://localhost:8080

MACHINE_URL=localhost
EOF
```

10.2.10 Run shell script to setup machine IP

```
./setup-machine-ip.sh
```

10.2.11 Run application with Terraform

```
terraform init
terraform apply -auto-approve
```

11 Tests

To ensure the reliability, maintainability, and robustness of the Marketing Automation Platform, a comprehensive testing strategy was implemented. This strategy follows a multi-layered approach, verifying the system's behavior at different levels of granularity, from individual logic components to full user journey simulations.

11.1 Unit Testing

Unit tests constitute the foundation of our testing strategy. The primary objective at this level was to validate the internal logic of individual components in isolation, ensuring that specific blocks of code perform as expected without the overhead of external dependencies.

We focused our unit testing efforts on three key layers:

- **Entities:** Verified domain logic, state transitions, and data validation rules within the core business objects.

- **Repositories:** Tested data access layers to ensure that database mapping and custom queries retrieve and persist data correctly.
- **Services:** Validated business logic and rules. Dependencies were mocked to isolate the service layer, allowing us to test edge cases and error handling mechanisms effectively.

11.2 Integration Testing

Moving up the testing hierarchy, integration tests were developed to verify the interactions between different layers of the application, with a specific focus on the **Controllers**.

These tests ensure that the REST API endpoints function correctly when exposed to HTTP requests. They validate:

- The correct mapping of HTTP verbs (GET, POST, PUT, DELETE) to controller methods.
- Accurate serialization and deserialization of JSON payloads.
- The proper HTTP status codes are returned (e.g., 200 OK, 404 Not Found, 400 Bad Request).
- The successful integration between the Controller layer and the Service layer.

11.3 End-to-End (E2E) Testing

To validate the system from the end-user's perspective, End-to-End (E2E) tests were implemented using a Behavior-Driven Development (BDD) approach.

We utilized **Cucumber** combined with **Selenium** to automate these tests:

- **Cucumber:** Allowed us to write test scenarios in Gherkin syntax (Given-When-Then), making the test cases readable and understandable for both technical and non-technical stakeholders.
- **Selenium WebDriver:** Served as the automation engine, simulating real user interactions within a web browser (clicking buttons, filling forms, and navigating pages).

These tests cover critical user journeys, such as logging in, creating a marketing campaign, and managing leads ensuring that the Frontend, Backend, and Database components work seamlessly together in a production-like environment.

12 Production Configuration

12.1 Environmental Variables

The system relies on a set of environmental variables to configure and control its components across development, testing, and production environments. These variables define essential parameters such as service ports, authentication credentials, API keys, feature flags, and resource scaling options. Centralizing these settings ensures consistent behavior, simplifies deployment, and allows sensitive information to be managed securely without hardcoding values in the application code. Table 2 provides a comprehensive overview of the key environmental variables used throughout the platform, along with their associated components and descriptions.

Table 2: Environmental Variables Configuration

Variable Name	Component	Description
`KEYCLOAK_IMAGE`	Keycloak	Docker image version for Keycloak authentication server
`KEYCLOAK_ADMIN`	Keycloak	Admin username for Keycloak console
`KEYCLOAK_ADMIN_PASSWORD`	Keycloak	Admin password for Keycloak console
`KEYCLOAK_LOCAL_PORT`	Keycloak	Host machine port to access Keycloak
`KEYCLOAK_DOCKER_PORT`	Keycloak	Internal Docker container port for Keycloak
`KEYCLOAK_REALM`	Keycloak	Authentication realm name
`KEYCLOAK_IMPORT_PATH`	Keycloak	Path to realm configuration files for import
`KC_HOSTNAME_URL`	Keycloak	Public URL for Keycloak server

Continued on next page

Table 2 – continued from previous page

Variable Name	Component	Description
SPRING_PROFILES_ACTIVE	Backend	Active Spring Boot profile (prod/dev/test)
SPRING_SECURITY_ISSUER_URI	Backend	OAuth2 token issuer URL for JWT validation
SPRING_SECURITY_JWK_URI	Backend	JWK Set URL for token signature verification
BACKEND_LOCAL_PORT	Backend	Host machine port to access backend API
BACKEND_DOCKER_PORT	Backend	Internal Docker container port for backend
backend_replicas	Backend	Number of backend container instances
POSTGRES_DB	Database	Database name
POSTGRES_USER	Database	Database username
POSTGRES_PASSWORD	Database	Database password
PG_LOCAL_PORT	Database	Host machine port to access PostgreSQL
PG_DOCKER_PORT	Database	Internal Docker container port for PostgreSQL
FRONTEND_LOCAL_PORT	Frontend	Host machine port to access frontend
FRONTEND_DOCKER_PORT	Frontend	Internal Docker container port for frontend
VITE_BACKEND_HOST	Frontend	Backend API URL for frontend to connect to
VITE_BACKEND_PORT	Frontend	Backend API port
VITE_KEYCLOAK_URL	Frontend	Keycloak URL for frontend authentication
VITE_UNLAYER_PUBLIC_KEY	Frontend	Unlayer email editor API key
frontend_replicas	Frontend	Number of frontend container instances
frontend_lb_external_port	Frontend	External port for frontend load balancer
frontend_lb_internal_port	Frontend	Internal port for frontend load balancer
backend_lb_external_port	Backend	External port for backend load balancer
backend_lb_internal_port	Backend	Internal port for backend load balancer
MAILJET_API_USERNAME	Email	Mailjet API key/username for sending emails
MAILJET_API_PASSWORD	Email	Mailjet API secret/password
VITE_FLAGSMITH_ENV_ID	Feature Flags	Flagsmith environment ID for feature flag management
VITE_API_VERSION	API	API version identifier
MACHINE_URL	Network	Public IP or domain name of the host machine
otel_exporter_otlp_endpoint	Observability	Elasticsearch APM endpoint for telemetry data
otel_exporter_otlp_headers	Observability	Authentication headers for APM (base64 encoded)
gemini_api_key	AI	Google Gemini API key for AI features
google_project_id	AI	Google Cloud project ID
facebook_access_token	Facebook	Facebook Graph API access token
facebook_app_id	Facebook	Facebook App ID
facebook_app_secret	Facebook	Facebook App Secret key
facebook_page_id	Facebook	Facebook Page ID (internal)
facebook_public_page_id	Facebook	Facebook Page ID (public-facing)
hubspot_access_token	HubSpot	HubSpot API access token for CRM integration

12.2 Build Process

The build process generates immutable, production-ready Docker images using a **container-first** approach with **multi-stage builds**. The backend is compiled with Maven and OpenJDK 17, producing a JAR that is deployed in a lightweight runtime image. The frontend is built with Node.js and Vite, producing static assets served via an Nginx container. This ensures consistent behavior across development, testing, and production, requiring only runtime environment variables to operate.

12.3 CI/CD Pipeline Architecture

The project utilizes a Continuous Integration and Continuous Deployment (CI/CD) strategy implemented via **GitHub Actions** on self-hosted runners. This configuration provides granular control over the build environment. Lifecycle management is handled via **Terraform** for Infrastructure as Code (IaC), with sensitive configuration securely injected via GitHub Secrets.

12.3.1 Continuous Integration (CI)

The CI pipeline (`ci_docker_compose_and_tests.yaml`) executes upon every Pull Request. Distinct from traditional CI processes that rely solely on unit tests, this pipeline provisions a **transient, full-stack environment** for every execution to ensure comprehensive system validation.

The runner initializes the required toolchain (Docker Buildx, Terraform, JDK 17) and deploys the complete infrastructure stack using `terraform apply`, comprising the Spring Boot backend, Vite frontend, PostgreSQL database, and Keycloak authentication server.

Prior to test execution, the pipeline performs **Health Verifications** to ensure service availability:

- **Backend:** Polls the `/actuator/health` endpoint (timeout: 200 s).
- **Frontend:** Verifies connectivity via the load balancer.
- **Keycloak:** Validates accessibility of the authentication realm.

Upon successful health checks, the full Maven test suite (unit, integration, and E2E) is executed. The process concludes with `terraform destroy` to eliminate resource drift and ensure a clean state for subsequent runs.

12.3.2 Test Reporting and Observability

To monitor code quality and regression, the project integrates the **Allure Framework**. Executed on the `master` branch, this process retains execution history to facilitate the detection of intermittent ("flaky") tests.

The resulting reports offer granular visibility into system stability, featuring:

- Stack traces for failure analysis.
- Execution timelines and duration metrics.
- Automated capture of logs and screenshots during failure events.

Reports are published to **GitHub Pages** to maintain a persistent record of project stability.

12.3.3 Continuous Deployment (CD)

The deployment pipeline (`deploy.yaml`) manages delivery to the production server, triggering automatically upon merges to the `master` branch.

The process prioritizes environment consistency. It initiates by terminating existing services and pruning unused Docker artifacts to optimize resource allocation. Subsequently, `terraform apply` is invoked to reconcile the production infrastructure with the defined configuration state.

12.4 Infrastructure Notes

The system infrastructure is managed primarily with **Terraform**, which provisions the core application services, including the backend, frontend, PostgreSQL database (with the `pgvector` extension), and Nginx-based load balancers. Persistent storage for the database is ensured via a dedicated Docker volume. All core services are deployed as Docker containers and connected through a shared Docker network.

Nginx acts as a reverse proxy and load balancer for both backend and frontend services, supporting horizontal scaling by running multiple container replicas as needed. Health checks and restart policies are defined for all critical containers to improve resilience and reliability.

Configuration and environment variables are managed through a combination of `terraform.tfvars` (for Terraform-managed services) and `.env` files (for Docker Compose-managed services), with the `setup-machine-ip.sh` script used to automate environment alignment for remote deployments. Sensitive values are injected securely using Terraform variables and GitHub Secrets.

Configuration and environment variables are managed through a combination of `terraform.tfvars` (for Terraform-managed services) and `.env` files (for Docker Compose-managed services), with the `setup-machine-ip.sh`

script used to automate environment alignment for remote deployments. Sensitive values are injected securely using Terraform variables and GitHub Secrets.

The platform integrates **OpenTelemetry** for observability, exporting traces and metrics to Elastic APM, which supports performance monitoring and troubleshooting across distributed components.

13 Feature Flag Management with Flagsmith

The platform integrates **Flagsmith** as the central mechanism for managing feature flags, enabling fine-grained control over how and when functionalities become available to users. This approach provided flexibility to activate or disable features without requiring new deployments, supporting safer iteration and controlled experimentation throughout development.

Two main categories of feature flags were used:

- **Standard feature toggles**, applied to enable or hide new functionalities while they were still being developed, tested, or validated.
- **A/B testing flags**, used to compare different versions of existing features, allowing the team to evaluate user experience variations across controlled user groups.

Standard feature toggles made it possible to gradually introduce new functionality into the system while ensuring that incomplete or unstable components remained inaccessible to end users. This allowed the team to deploy work in progress safely, maintaining platform stability even as new modules were being implemented.

A/B testing flags were used to evaluate alternative versions of existing behaviour, such as layout or interaction refinements. Flagsmith ensured that each user consistently received the same variant across sessions, enabling reliable comparison and data-driven decisions regarding interface improvements.

After successful authentication through Keycloak, the application identified each user in Flagsmith. This ensured consistent flag behaviour per user and supported targeted rollouts, such as enabling features only for internal testers or specific user groups.

14 Observability

This section describes the design decisions and implementation steps taken to introduce observability into the system, covering logs, traces, and metrics across both the frontend and backend layers.

The first step involved ensuring that all running services exposed reliable health indicators. Health checks were added to every container in the platform, regardless of whether the container was provisioned through **Terraform** (Backend, Frontend, PostgreSQL, and NGINX load balancers) or through **Docker Compose** (Keycloak). Establishing these health checks provided a foundational visibility layer during development, allowing the team to continuously verify the operational status of each service, detect early-stage failures, and confirm that dependencies were correctly initialized before traffic was routed to them.

This was an essential prerequisite for the remaining observability pipeline: by guaranteeing that each container reported a valid readiness or liveness state, the infrastructure could reliably assess service stability before integrating more advanced telemetry mechanisms such as distributed tracing, log correlation, and metrics exportation.

14.1 Backend Observability

For the backend, three primary categories of observability signals were implemented: **application logs**, **distributed traces**, and **system and application metrics**. Together, these provide full visibility into how the Spring Boot application behaves internally and how it interacts with external services and downstream dependencies.

- **Application Logs** are emitted in Elastic Common Schema (ECS) format, enabling structured ingestion into Elasticsearch. Each log entry automatically includes correlation fields such as `trace.id` and `span.id`, which link backend logs directly to frontend and backend traces.
- **Distributed Traces** are generated automatically through OpenTelemetry instrumentation. These traces capture every inbound HTTP request, internal service call, database access, scheduled task execution, and outbound request to third-party APIs such as MailJet, Facebook Graph API, and HubSpot.

- **Metrics** include JVM memory and garbage collection statistics, CPU usage, database connection pool utilisation, and HTTP request latency. These are emitted through Micrometer's OpenTelemetry bridge and provide insight into the runtime behaviour and resource usage of the backend.

All backend telemetry (logs, traces, and metrics) is exported in OTLP format by the OpenTelemetry Java agent and forwarded to the **OpenTelemetry Collector**, deployed as a dedicated Docker service in the same network. The collector processes and enriches the telemetry before forwarding it to Elastic Cloud, where:

- **Elastic APM** stores and visualises the distributed traces,
- **Elasticsearch** indexes the structured logs and time-series metrics,
- **Kibana** provides dashboards, correlation views, and alerting capabilities.

This backend observability pipeline supports root-cause analysis of failures or latency spikes and provides operational visibility into the health and performance of all backend services.

14.2 Frontend Observability

For the frontend, three categories of telemetry events were implemented: **page load events**, **API call events**, and **user interaction events**. These categories were chosen to provide full visibility into how users navigate and interact with the system.

- **Page Loads** are emitted whenever a user navigates to a new route within the React application. This enables analysis of navigation patterns and page-level performance.
- **API Calls** are traced whenever the frontend communicates with the backend through the NGINX load balancer. These traces allow the system to correlate frontend requests with backend spans, creating end-to-end distributed traces.
- **User Actions** capture meaningful interactions, such as clicking buttons, submitting forms, or triggering workflow operations. These events help identify usage patterns and support usability analysis.

All frontend events are generated through the OpenTelemetry Web SDK and exported in OTLP format to the **OpenTelemetry Collector**, running as a dedicated Docker service. The collector then forwards these traces to Elastic Cloud, where they are ingested by **Elastic APM** and stored in **Elasticsearch**. Once stored, the traces become available in **Kibana** through visual dashboards, allowing the team to analyse user behaviour, correlate frontend spans with backend operations, and diagnose latency issues or UI misbehaviour. This completes the observability pipeline for the frontend and provides a comprehensive view of the user experience across the entire system.

14.3 SLOs/SLIs

Although several Service Level Objectives (SLOs) were defined during the design phase, only four were ultimately added to the Elastic Cloud dashboard for continuous monitoring. These indicators played a crucial role in assessing whether the system was behaving as expected and whether its performance remained within the thresholds considered acceptable. The four implemented SLOs are described below:

- **POST Latency**: This SLO ensures that at least 95% of all POST requests complete within 600 ms. This indicator allows the team to evaluate how responsive the system feels during user operations such as creating campaigns, emails, or workflows, and to verify that server-side processing remains performant under normal load.
- **User Interaction Responsiveness**: This performance-oriented SLO ensures that at least 99% of all user actions in the frontend (the previously described User Action events) are processed in under 50 ms. Monitoring this metric confirms that the interface remains highly responsive and aligned with modern user experience expectations.
- **Frontend Navigation Latency**: This SLO requires that at least 95% of all spans generated during frontend page navigation complete within 650 ms. This metric evaluates whether users are able to navigate through the application smoothly, without perceiving delays or rendering bottlenecks.

- **API Availability**: The final SLO focuses on ensuring the overall availability of the backend service. Here, the objective is to guarantee that 99% or more of all API requests (including POST, GET, DELETE, and PUT) succeed without unexpected errors. This metric provides a direct measure of system reliability from the perspective of API consumers.

The following figure presents the results obtained from the Elastic Cloud SLO dashboard. As shown, the system satisfied all four defined objectives, indicating that, according to the monitored metrics, the system maintained the expected levels of performance and reliability.

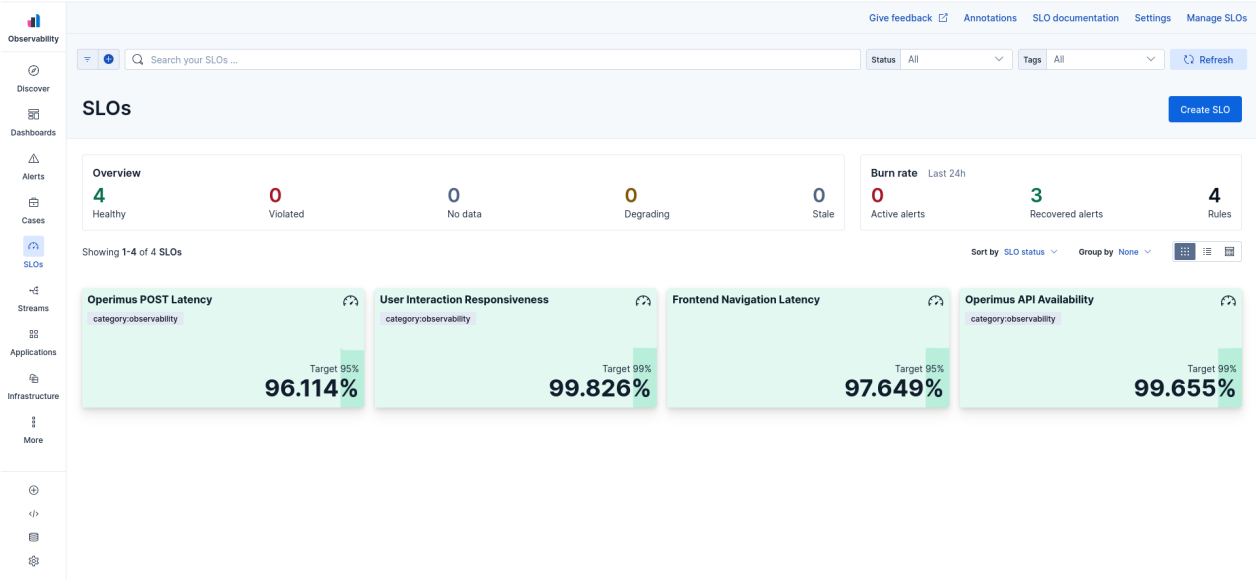

Figure 5: Service Level Objectives

15 Development Workflow and Code Review Policies

The team follows a structured workflow to ensure consistency, maintainability and quality across all development activities. This workflow defines how the backlog is organised, how branches are created, how pull requests are handled and how code reviews are performed.

15.1 Backlog Organisation

The backlog is managed through GitHub Issues and is used to track all tasks, features and bug reports. Each developer must ensure that the work they intend to carry out is properly documented:

- Every task must have an associated Issue before development begins.
- Developers create the Issue if it does not already exist and assign it to themselves.
- Each Issue must be added to the project board and linked to the current iteration.
- Issues must include the appropriate labels, indicating their category (e.g., *new*, *bug*, *documentation*, *enhancement*), area (e.g. *frontend*, *backend*), effort (e.g., *easy*, *hard*, *medium*).
- Issues serve as the single source of truth for discussion, scope definition and progress tracking.
- The Scrum Master is responsible for preparing and maintaining the meeting minutes for each sprint ceremony, ensuring that decisions, action points and blockers are clearly documented and reflected in the backlog when necessary.

15.2 Pull Requests and Templates

To maintain consistency, all pull requests follow predefined templates. These templates vary depending on the type of work being submitted, such as features, bug fixes, refactors, hotfixes or documentation updates.

- All branches are created from the `dev` branch.
- Branch naming conventions follow the pattern:
  - `feature/<name>`
  - `bugfix/<name>`
  - `hotfix/<name>`
  - `refact/<name>`
  - `docs/<name>`
- Pull requests must use the appropriate template available in the `.github` directory. The template can be selected by adding the query parameter `&template=template_name.md` to the Pull Request creation URL.
- Only the DevOps engineer is permitted to merge pull requests into `dev` or `main`.

15.3 Code Review Rules

Code reviews are a mandatory step in the workflow and help maintain high code quality standards across the project.

- Every pull request must be reviewed and approved by at least **one other team member**.
- Authors cannot approve or merge their own pull requests.
- Reviews must check:
  - Code readability, structure and maintainability.
  - Alignment with coding standards and project architecture.
  - Presence of adequate tests (unit or integration) for new or changed functionality.
  - That no sensitive information (e.g., credentials, tokens) is committed.
- Pull requests containing insufficient or missing tests will not be approved.
- Test coverage must not decline compared to the current project baseline, with emphasis on meaningful test cases rather than coverage metrics alone.

15.4 CI/CD Validation

Before a pull request can be merged, it must successfully pass all CI/CD pipeline checks. These automated tests guarantee that new changes do not introduce regressions or inconsistencies into the codebase.

- CI/CD validates code format, linting, unit tests, integration tests and build correctness.
- Pull requests that fail CI/CD checks cannot be merged.
- Only after all checks are green and the review process is completed can a pull request be considered for merging.

15.5 Final Merge Policy

Once a pull request is approved and passes all CI/CD tests:

- The DevOps engineer performs the merge into the appropriate branch.
- Release pull requests targeting the `main` branch require additional validation from the Product Owner.